



Project Proposal: Titan Track

An OOP-Driven Command Line Fitness Ecosystem with Local Data Persistence & Automated Test Architecture

Course:	Object-Oriented Programming (OOP)	Date:	June 2, 2026
Developer 1:	Zarak Khan Afridi Reg No: B25F0485CYS123	Target Env:	Windows Console (CMD/ PowerShell)
Developer 2:	Amar Sadat (Reg No: B25F2099CYS127	Compiler Standard:	ISO C++11 / C++14 Compliant

1. EXECUTIVE SUMMARY

In modern software design courses, moving from abstract theoretical definitions to functional applications remains a key milestone. **Titan Track** is designed as a secure, local, high-performance command-line interface (CLI) application for Windows environments. It enables users to securely log, manage, and audit their fitness milestones.

Rather than relying on volatile, short-lived runtime memory, Titan Track implements local file streams (*std::fstream*) to provide permanent storage. The application serves two core purposes: delivering a highly efficient console utility for personal tracking, and implementing strict Object-Oriented Programming (OOP) standards alongside an integrated unit testing suite.

2. SYSTEM CAPABILITIES & IMPLEMENTATION ARCHITECTURE

2.1 Gatekeeping Framework & Strict Validation Engine

To prevent unauthorized file reads or data mix-ups, the authentication engine enforces structured data constraints directly within the user-creation pipeline:

- **Email Syntax Validation:** Scans string inputs using an explicit state-check to confirm the presence of both an '@' character and a subsequent domain period (('.'), preventing poorly formed entries.
- **Password Complexity Rules:** Rejects weak input choices by checking character groups during input. Passwords must be at least **6 characters** long and contain at least one uppercase letter (*A-Z*), one lowercase letter (*a-z*), and at least one non-alphanumeric special character symbol (such as *!, @, #, \$, %, ^, &, *, _, +*).
- **Double-Blind Confirmation:** Enforces dual-entry verification during configuration to block typos before

writing to the local system database.

2.2 Persistent Flat-File Simulation Engine

Data persistence is achieved without the configuration overhead of database servers. Instead, Titan Track handles file-level operations using structured comma-separated data streams:

Local Storage Database	Data Layout Fields	Internal Operational Domain
users.txt	[Email],[Password]	Tracks verified security records for real-time authentication loops.
workouts.txt	[Email],[Name],[Weight],[Sets],[Reps],[Dur],[Type],[Time],[Status]	Appends comprehensive workout logs, linked to the specific authenticated email.

2.3 User Dashboard & Operational Loop

Upon authentication, the user session initializes a dashboard pointer that unlocks the core functional loop:

- 1. Polymorphic Workout Tracking:** Allows users to log strength records (capturing attributes like absolute weights, target sets, and repetitions) or cardio milestones (tracking duration values in minutes).
- 2. Targeted Audit Stream:** Reads through the flat-file logs sequentially, parsing rows using string tokens to isolate and display records matching the active user.
- 3. Routine Completion Toggle:** Allows users to change state flags from pending to finished, editing individual activity statuses natively in the system.

3. MAPPING TO OBJECT-ORIENTED PILLARS

The code structure is explicitly decoupled into dedicated class systems, applying the four core pillars of Object-Oriented Programming:

1. Encapsulation

All core instance states (such as credentials and raw exercise vectors) are declared within `private` or `protected` visibility boundaries. Direct state manipulation from the global scope or `main()` function is completely barred. Component communication is mediated through explicit accessor methods.

2. Inheritance

The architecture segments user profiles through structured tiers. A parent base class, `User`, encapsulates universal access details and credential validation logic. A specialized subclass, `TitanUser`, publicly inherits these credentials while extending the class model with dashboard operations and custom routine collection structures.

3. Polymorphism

The system showcases compile-time polymorphism through method overloading within `TitanUser`, handling strength metrics and cardio logs based on input signatures. Runtime polymorphism is established through an abstract `View` interface, which forces child screens (`LoginScreen`, `DashboardScreen`) to implement their own rendering behaviors.

4. Abstraction

File handling logic, string tokenization operations, and console display formats are decoupled from user interfaces. An isolated utility class, `StorageManager`, manages system-level interactions like validating file access states, exposing clean, simple interfaces to the broader application framework.

4. TECHNICAL IMPLEMENTATION BLUEPRINT

The implementation structure matches the formal class configurations below, ensuring clear architectural design:

```

#ifndef TITAN_TRACK_BLUEPRINT
#define TITAN_TRACK_BLUEPRINT

#include <string>
#include <vector>

// Abstract Base View layer displaying Runtime Polymorphism
class View {
public:
    virtual void displayMenu() = 0; // Pure virtual enforcement
    virtual ~View() {}
};

// Domain Object illustrating Encapsulation
class Workout {
private:
    std::string userEmail;
    std::string exerciseName;
    double weight;
    int sets, reps, durationMinutes;
    std::string type; // "Strength" or "Cardio"
    std::string status; // "Pending" or "Completed"
public:
    Workout(std::string em, std::string name, std::string t);
    std::string toCSVLine() const;
    void markAsCompleted() { status = "Completed"; }
};

// Abstracted Manager for Local File Persistence Database
class StorageManager {
public:
    static bool ensureFilesExist();
    static bool writeUserRecord(const std::string& email, const std::string& password);
    static bool fetchUserRecords(const std::string& email, std::string& outPassword);
    static void appendWorkoutLog(const Workout& log);
    static std::vector<Workout> parseUserLogs(const std::string& email);
};

#endif

```

5. QUALITY ASSURANCE & EMBEDDED TESTING FRAMEWORK

To ensure stability before deployment, Titan Track includes an automated unit-testing pipeline. Using a split-compilation flag system (`#ifndef RUNNING_TESTS`), developers can launch a dedicated test runner that uses strict assertions to verify critical systems:

- **Email Edge Verification:** Programmatic tests check both compliant addresses and faulty samples (e.g., missing symbols, empty usernames, or broken domains) to ensure input safety.
- **Password Criteria Evaluation:** Tests password verification checks against short, uniform, or symbol-free strings, verifying that clear error messages are generated for weak inputs.
- **Polymorphic Logging Verification:** Simulates active user file operations to confirm that both Strength and Cardio variations process accurately throughout the parsing system.